

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 16 · Estructuras de datos dinámicas: pilas, colas y listas enlazadas

ArrayList vs LinkedList: costes de acceso e inserción

Pilas (LIFO) y colas (FIFO): ArrayDeque y Deque

Listas enlazadas: nodos, inserción y borrado

Iteradores: Iterator, for-each y ConcurrentModificationException

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 3
Valoración	Estructuras de datos dinámicas

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué clase de Java implementa una lista doblemente enlazada?
- a) ArrayList
 - b) LinkedList
 - c) ArrayDeque
 - d) Vector

RESPUESTA / SOLUCIÓN

b) LinkedList — Está implementada como lista doblemente enlazada; cada nodo guarda referencia al anterior y al siguiente.

-
2. ¿Qué estructura sigue el principio FIFO (primero en entrar, primero en salir)?
- a) Pila
 - b) Cola
 - c) Lista
 - d) Conjunto

RESPUESTA / SOLUCIÓN

b) Cola — En una cola el primer elemento encolado es el primero en salir (FIFO).

-
3. ¿Qué estructura sigue el principio LIFO (último en entrar, primero en salir)?
- a) Pila
 - b) Cola

- c) Deque
- d) Vector

RESPUESTA / SOLUCIÓN

- a) Pila — En una pila el último elemento apilado es el primero en salir (LIFO).

4. ¿Qué método de Deque añade un elemento al final de la estructura?

- a) addLast()
- b) addFirst()
- c) push()
- d) insert()

RESPUESTA / SOLUCIÓN

- a) addLast() — Añade el elemento al final; addFirst() y push() lo añaden al principio.

5. ¿Qué método de Deque extrae y devuelve el primer elemento (comportamiento de cola)?

- a) pollFirst()
- b) pop()
- c) peekLast()
- d) removeLast()

RESPUESTA / SOLUCIÓN

a) pollFirst() — Extrae el primer elemento (o poll(), equivalente); pop() extrae del principio con comportamiento de pila.

-
6. **ArrayList es más eficiente que LinkedList para accesos aleatorios por índice (get(i)).**

RESPUESTA / SOLUCIÓN

Verdadero — ArrayList accede por índice en $O(1)$ al estar respaldado por un array; LinkedList debe recorrer nodos hasta la posición ($O(n)$).

-
7. **¿Qué interfaz permite recorrer una colección con hasNext() y next()?**
- a) Iterator
 - b) Iterable
 - c) Enumerator
 - d) Loopable

RESPUESTA / SOLUCIÓN

a) Iterator — El bucle for-each usa internamente un Iterator sobre la colección (Iterable).

8. Modificar una lista con `remove()` mientras se recorre con `for-each` lanza `ConcurrentModificationException`.

RESPUESTA / SOLUCIÓN

Verdadero — El iterador interno detecta modificaciones estructurales; hay que usar `iterator.remove()` o copiar la lista.

-
9. ¿Qué método usa la clase `Stack` (`java.util`) para añadir un elemento encima?
- a) `push()`
 - b) `addTop()`
 - c) `insert()`
 - d) `put()`

RESPUESTA / SOLUCIÓN

a) `push()` — `Stack` (clase legacy) usa `push()` para apilar y `pop()` para desapilar.

-
10. ¿Cuál es la implementación recomendada de pila en Java moderno?
- a) `ArrayDeque`
 - b) `Stack`
 - c) `Vector`
 - d) `ArrayList`

RESPUESTA / SOLUCIÓN

a) ArrayDeque — Más rápida que Stack (que es sincronizada y hereda de Vector) y no limitada por el tamaño del array.

Parte 2: Análisis y depuración

3 puntos

1. El siguiente código implementa una cola con ArrayList. ¿Qué problema de rendimiento tiene y cómo lo mejorarías? (1,5 pts)

```
import java.util.ArrayList;\npublic class ColaConLista {\n    private ArrayList<String> datos = n
```

RESPUESTA / SOLUCIÓN

Problema: `datos.remove(0)` desplaza todos los elementos una posición hacia la izquierda → $O(n)$ por cada desencolado.

Con n operaciones el coste total es $O(n^2)$, inaceptable para colas grandes.

Solución: usar `ArrayDeque<String>` (o la interfaz `Queue`) con `addLast()/pollFirst()`, que operan en $O(1)$ amortizado.

-
2. La siguiente pila usa un array fijo. ¿Qué ocurre al apilar más elementos de los permitidos y cómo se corrige? (1,5 pts)

```
public class PilaFija {\n    private int[] datos = new int[10];\n    private int tope = -1;\n}
```

RESPUESTA / SOLUCIÓN

Al apilar el elemento 11 se accede a `datos[10]`, que está fuera del array (índices 0-9) → `ArrayIndexOutOfBoundsException`.

Además `pop()` sobre una pila vacía devuelve `datos[-1]` sin control.

Solución: usar `ArrayDeque<Integer>` (`push/pop`) o redimensionar el array manualmente (`Arrays.copyOf`) y comprobar `tope` antes de operar.

Parte 3: Ejercicios de programación

6 puntos

1. Implementa una clase ColaClientes que gestione una cola de nombres con ArrayDeque: encolar(), desencolar() y mostrar() (recorrido sin vaciar). Crea un main de prueba. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayDeque;
import java.util.Deque;

public class ColaClientes {

    private Deque<String> cola = new ArrayDeque<>();

    public void encolar(String nombre) { cola.addLast(nombre); }

    public String desencolar() { return cola.pollFirst(); }

    public void mostrar() {
        for (String c : cola) System.out.println(c);
    }

    public static void main(String[] args) {
        ColaClientes c = new ColaClientes();
        c.encolar("Ana"); c.encolar("Luis"); c.encolar("Marta");
        System.out.println("Atendido: " + c.desencolar()); // Ana
        c.mostrar(); // Luis, Marta
    }
}
```

2. Escribe un programa que compruebe si una expresión con paréntesis (), corchetes [] y llaves {} está balanceada, usando una pila. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayDeque;
import java.util.Deque;

public class ParentesisBalanceados {

    static boolean balanceado(String expr) {
        Deque<Character> pila = new ArrayDeque<>();
        for (char c : expr.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') pila.push(c);
            else if (c == ')' || c == ']' || c == '}') {
                if (pila.isEmpty()) return false;
                char apertura = pila.pop();
                if (!coinciden(apertura, c)) return false;
            }
        }
        return pila.isEmpty();
    }

    static boolean coinciden(char a, char c) {
        return (a == '(' && c == ')') || (a == '[' && c == ']') || (a == '{' && c == '}');
    }

    public static void main(String[] args) {
        System.out.println(balanceado("(a + b) * [c - {d}]")); // true
        System.out.println(balanceado("(a + b]"));           // false
    }
}
```

3. } Implementa una lista enlazada simple propia: clase Nodo (valor y siguiente) y clase ListaEnlazada con métodos agregar(int) e imprimir(). (2 pts)

RESPUESTA / SOLUCIÓN

```
class Nodo {
    int valor;
    Nodo siguiente;
    Nodo(int valor) { this.valor = valor; }
}

public class ListaEnlazada {
    private Nodo cabeza;

    public void agregar(int v) {
        Nodo nuevo = new Nodo(v);
        if (cabeza == null) { cabeza = nuevo; return; }
        Nodo actual = cabeza;
        while (actual.siguiente != null) actual = actual.siguiente;
        actual.siguiente = nuevo;
    }

    public void imprimir() {
        for (Nodo n = cabeza; n != null; n = n.siguiente)
            System.out.print(n.valor + " ");
        System.out.println();
    }

    public static void main(String[] args) {
        ListaEnlazada l = new ListaEnlazada();
        l.agregar(3); l.agregar(7); l.agregar(1);
        l.imprimir(); // 3 7 1
    }
}
```

Parte 4: Pregunta teórica

2 puntos

1. Compara ArrayList y LinkedList indicando en qué operaciones es mejor cada una. ¿Cuándo elegirías ArrayDeque?

RESPUESTA / SOLUCIÓN

ArrayList: acceso por índice $O(1)$, inserción/borrado al final $O(1)$ amortizado, pero insertar/borrar en el centro $O(n)$ por el desplazamiento. Mejor para lecturas frecuentes por índice.

LinkedList: inserción/borrado en posiciones conocidas $O(1)$ (si se tiene el nodo), pero acceso por índice $O(n)$. Mejor para muchas inserciones/borrados al principio.

ArrayDeque: ideal para pilas y colas (push/pop, addFirst/addLast, pollFirst/pollLast) con $O(1)$ amortizado, sin el coste de los nodos de LinkedList.

Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)